

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

Um **grafo** é um par $G = (V, E)$ onde

- V é um conjunto finito de elementos chamados **vértices**.
- E é um conjunto finito de pares de vértices, chamados **arestas**.

Ex.: $V = \{a, b, c, d, e\}$,
 $E = \{(a, b), (a, c), (b, c), (b, d), (c, d), (c, e), (d, e)\}$.

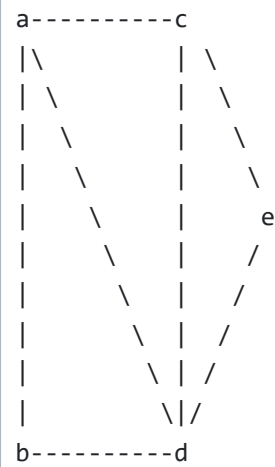
Dada uma aresta $e = (a, b)$, dizemos que os vértice a e b são **extremos** da aresta e , e que a e b são **adjacentes**. Dizemos também que a e b são **incidentes** à aresta e , e que e é **incidente** aos vértices a e b .

Um grafo é **simples** quando não possui laços e arestas múltiplas.

- Laço: aresta com extremos idênticos.
- Arestas múltiplas: 2 ou mais arestas com os mesmos extremos. Se as pares de vértices nas arestas são ordenados, o grafo é **direcionado**. Caso contrário, o grafo é **não direcionado**.

Representação por matriz de adjacência

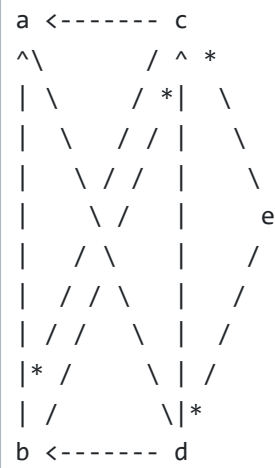
Grafo não direcionado



```
#define N 5

char A[N][N] = {
    0,1,1,0,0, // a
    1,0,1,1,0, // b
    1,1,0,1,1, // c
    0,1,1,0,1, // d
    0,0,1,1,0  // e
// a b c d e
};
```

Grafo direcionado

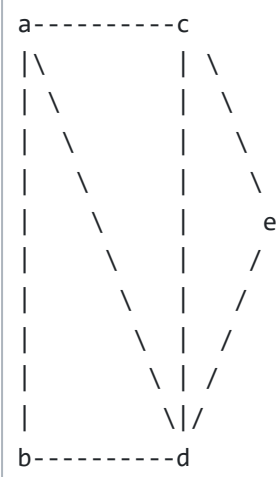


```
#define N 5

char A[N][N] = {
    0,0,0,0,0, // a
    1,0,1,0,0, // b
    1,1,0,0,0, // c
    0,1,1,0,0, // d
    0,0,1,1,0  // e
    // a b c d e
};
```

Representação por lista de adjacência

Grafo não direcionado



```
+---+
| a|--> b -> c
| b|--> a -> c -> d
| c|--> a -> b -> d -> e
| d|--> b -> c -> e
| e|--> c -> d
+---+
```

```
#include <list>
enum Vertices {a,b,c,d,e};
list<int> Adj[N];
Adj[a].push_back(b);
Adj[a].push_back(c);
Adj[b].push_back(a);
Adj[b].push_back(c);
Adj[b].push_back(d);
Adj[c].push_back(a);
Adj[c].push_back(b);
Adj[c].push_back(d);
Adj[c].push_back(e);
Adj[d].push_back(b);
Adj[d].push_back(c);
Adj[d].push_back(e);
Adj[e].push_back(c);
Adj[e].push_back(d);
```

Grafo direcionado

```
a <----- c
^ \      / ^ *
| \      / *| \
| \      / / | \
| \    / / / | \
| \  \ / / |   \
|   \ \ / |     e
|   / \   |     /
|  / / \   |    /
| / /   \  |   /
|* /     \ |  /
| /       \|*
b <----- d
```

```
+---+
| a-|-->
| b-|--> a -> c
| c-|--> a -> b
| d-|--> b -> c
| e-|--> c -> d
+---+
```

```
#include <list>
enum Vertices {a,b,c,d,e};
list<int> Adj[N];
Adj[b].push_back(a);
Adj[b].push_back(c);
Adj[c].push_back(a);
Adj[c].push_back(b);
Adj[d].push_back(b);
Adj[d].push_back(c);
Adj[e].push_back(c);
Adj[e].push_back(d);
```

Exemplo: Gerando grafos direcionados aleatórios

```
#include <stdlib.h>
#define DENSIDADE 0.1

void grafo_aleat_dir_mat() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            A[i][j] = ((double) rand() / RAND_MAX) < DENSIDADE;
}

void grafo_aleat_dir_lst() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            if (((double) rand() / RAND_MAX) < DENSIDADE)
                Adj[i].push_back(j);
}

// Inicializa semente do gerador de num. aleatórios
srand(123456789);
```

Exemplo: Gerando grafos não direcionados aleatórios

```
void grafo_aleat_nao_dir_mat() {
    for (int i = 0; i < N; i++)
        for (int j = i; j < N; j++) {
            A[i][j] = ((double) rand() / RAND_MAX) < DENSIDADE;
            A[j][i] = A[i][j];
        }
}

void grafo_aleat_nao_dir_lst() {
    for (int i = 0; i < N; i++)
        for (int j = i; j < N; j++)
            if (((double) rand() / RAND_MAX) < DENSIDADE) {
                Adj[i].push_back(j);
                Adj[j].push_back(i);
            }
}
```

Exemplo: Copiando de uma representação para a outra

```
// Copia o grafo na matriz de adjacencias p/ a lista de adj.
// Assume listas de adjacências vazias.
void copia_grafo_mat_lst() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            if (A[i][j]) Adj[i].push_back(j);
}

// Copia o grafo na lista de adjacencias p/ a matriz de adj.
// Assume matriz inicializada com zeros.
void copia_grafo_lst_mat() {
    list<int>::iterator it;
    for (int i = 0; i < N; i++)
        for (it=Adj[i].begin(); it != Adj[i].end(); it++)
            A[i][*it] = 1;
}
```

A escolha da representação pode afetar a complexidade

Exemplo: Verificando se a aresta (100, 200) pertence ao grafo

```
// USANDO MATRIZ (mais rápido)
if (A[100][200]) ...

// USANDO LISTA
list<int>::iterator it;
for (it=Adj[100].begin(); it != Adj[100].end(); it++)
    if (*it == 200) ...
```

Exemplo: Determinando o grau do vértice 100

Grau: número de arestas que incidem no vértice.

```
// USANDO MATRIZ
int grau = 0;
for (int j = 0; j < N; j++)
    if (A[100][j]) grau++;

// USANDO LISTA (mais rápido)
int grau = 0;
list<int>::iterator it;
for (it=Adj[100].begin(); it != Adj[100].end(); it++)
    grau++;
// ALTERNATIVA
int grau = Adj[100].size();
```

Exercícios

1. Para obter o grafo transposto de G , denotado por G^T , invertamos a direção de cada aresta de G . Forneça pseudocódigo e complexidade de algoritmo para obter o grafo transposto de G .

Solução para lista adjacências: para cada vértice v na lista de adjacências de u (em G), insira u na lista de adjacências (em G^T). $O(V+E)$.

Solução para matriz adjacências: retorne a matriz transposta (inverter linhas e colunas). $O(V^2)$.

[◀ Notas de aula: ED para conjuntos disjuntos \(Union-find\)](#)

Seguir para...

[Vídeos da disciplina Desafios de Programação \(Prof. Wladimir\) sobre EDs ►](#)

 Baixar o aplicativo móvel.